

0. 前端原型总览

本章待原型截图上屏后补充。页面布局如下：



核心布局：搜索结果+个股卡片列表。复用现有 StockCard 组件，每张卡片展示完整的结构/阶段/买点/止损/信号信息。顶部筛选条控制信号过滤和排序。

1. 背景与问题

现状： - 已有涨幅榜（按指定区间涨幅排序） - 已有自选股/持仓的买点扫描 - 已有逻辑追踪系统

痛点： - 同花顺热股榜代表了全市场关注度最高的100只个股，是发现主线热点的重要信号源 - 当前系统只能从自选股+持仓范围内扫描买点，无法覆盖市场热点股的买点机会 - 每天复盘扫描才出3只自选股有买点，可能市场热点股中有更多机会被遗漏

目标： 将同花顺24h热股Top100接入系统，用3L体系分析每只热股的结构/阶段/买点/信号，让用户像翻自选股一样快速浏览全市场热点股的交易机会。

2. 设计思想

2.1 核心理念

热点即机会 — 市场关注度集中的个股是短线/波段机会的重要来源。将这些热股纳入3L分析框架，解决“自选股太少看不到全市场”的问题。

2.2 方案选择

候选方案	优点	缺点	结论
自建热点扫描（成交量/涨幅）	不受外部API限制	算法复杂、需要大量数据	✖

接入同花顺热股API+3L分析	数据直接可用、100只覆盖面广	依赖第三方API	✓
仅展示热股原始数据	简单快速	没有3L分析意义不大	✗

2.3 设计原则

- 1. 复用现有能力 — 使用已有的 StockCard 组件和 get_stock_card()，不重复造轮子
- 2. 信息密度优先 — 每张卡片展示结构/阶段/买点/止损/信号/主线定位，让用户一眼判断价值
- 3. 筛选优于排序 — 默认按热度排名排序，但提供信号/结构/阶段筛选快速定位买点

2.4 屏幕内外

- 做什么：展示同花顺热股Top100 + 每只个股的3L完整分析（结构/阶段/买点/信号/止损）
- 不做什么：不自己做热度排名计算、不做个股历史热度追踪（第一版不做热度曲线）
- 不做什么：不修改现有StockCard组件（直接复用）
- 不做什么：不缓存热股数据（每次请求实时拉取，热股数据天然时效性高）

3. 数据模型

3.1 同花顺API源数据结构

```
{
  "market": 17,
  "code": "601991",
  "name": "大唐发电",
  "rate": "59325682",
  "rise_and_fall": -3.19,
  "order": 1,
  "hot_rank_chg": 0,
  "tag": {
    "concept_tag": ["绿色电力", "风电"],
    "popularity_tag": "持续上榜"
  }
}
```

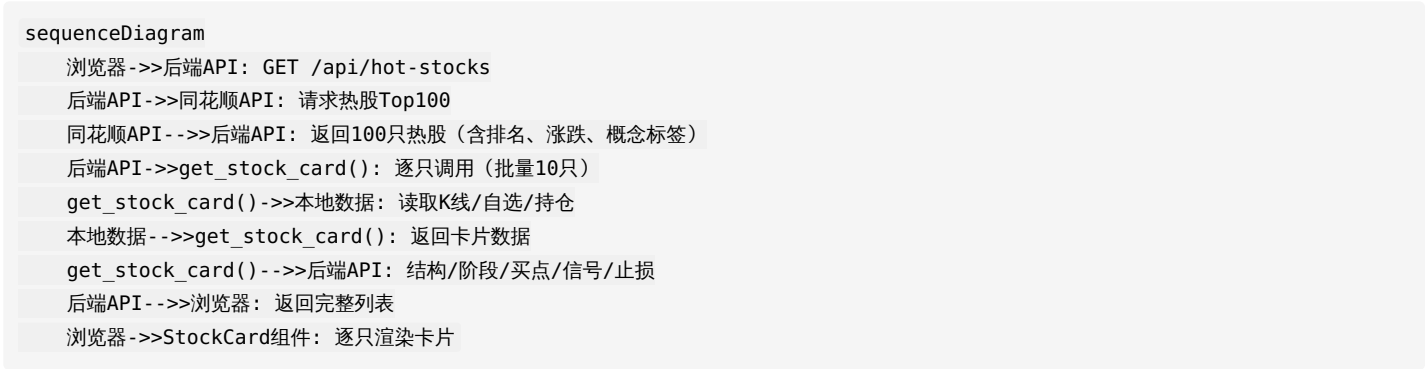
字段说明：- market: 交易所（17=上交所, 33=深交所）— 用于确定代码前缀(.SS/.SZ) - code: 6位股票代码 - rate: 热度值（字符串，单位为万？实际展示为整数） - rise_and_fall: 涨跌幅百分比 - order: 热度排名（1=最热） - tag.concept_tag: 概念标签数组 - tag.popularity_tag: 人气标签（如“首板涨停”、“3天2板”、“持续上榜”）

3.2 API返回数据

```
{
  "stocks": [
    {
      "code": "601991",
      "name": "大唐发电",
      "hot_rank": 1,
      "hot_value": 59325682,
      "change": -3.19,
      "price": null,
      "sector": "电力",
      "structure": "区间震荡",
      "stage": "区间顶部",
      "signal": "hold",
      "trading_system": "3l",
      "buy_point": null,
      "stop_loss": null,
      "stop_loss_pct": null,
      "mainline_level": "次级主线",
      "concept_tags": ["绿色电力", "风电"],
      "popularity_tag": "持续上榜",
      "triggered_signals": [...],
    }
  ]
}
```

```
    "fusion_type": "balance",
    "conclusion": "区间顶部受压..."
  }
],
"scan_time": "2026-06-04 10:38:00",
"total": 100
}
```

3.3 数据流



4. 系统设计

4.1 架构总览



4.2 核心逻辑

后端：1. 请求同花顺API获取热股Top100（带User-Agent和Referer防403） 2. 对每只股票调用 `get_stock_card()` 获取3L分析数据 3. 合并热股特有字段（`hot_rank`, `hot_value`, `concept_tags`, `popularity_tag`） 4. 返回完整列表

前端：1. 页面加载时请求 `/api/hot-stocks` 2. 使用 `StockCard` 组件渲染每只股票（保证与自选股/持仓一致的展示） 3. 筛选条控制：按信号（buy/hold/sell）、结构（上涨/震荡/下降）、阶段、主线级别 4. 排序切换：热度排名 / 涨跌幅 / 有无买点

4.3 API 设计

GET /api/hot-stocks

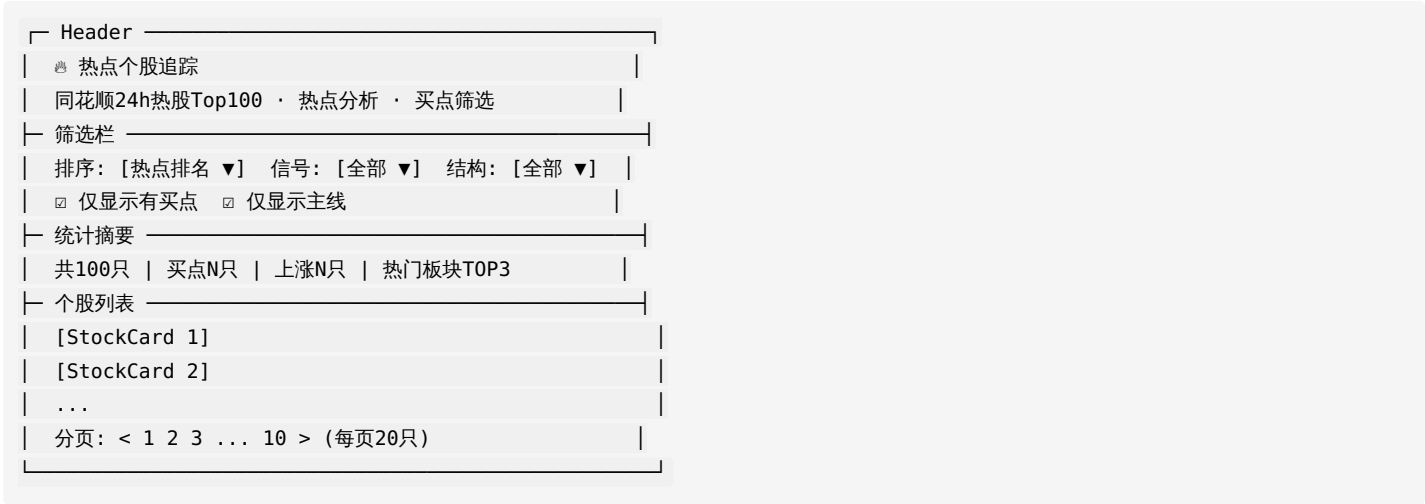
- 无参数（默认取24h热股Top100），后续可加 `type=day|hour`、`limit=N`
- 响应：{ `stocks`: `HotStockItem[]`, `scan_time`: `string`, `total`: `number` }
- 性能：后端获取100只+分析，预估3-5秒（含价格查询）

HotStockItem 扩展 **BuySignalItem**：

```
interface HotStockItem extends BuySignalItem {
  hot_rank: number          // 热度排名
  hot_value: number         // 热度值
  change: number            // 涨跌幅（从热股API来，比行情更新快）
  concept_tags: string[]    // 概念标签
  popularity_tag: string    // 人气标签（首板涨停、3天2板、持续上榜等）
}
```

4.4 前端设计

页面结构：



交互设计： - 默认按热度排名排序 — 热度最高的在最前面 - 筛选条件改变后实时过滤（不重新请求API） - “仅显示有买点” — 最常用的筛选，快速定位买入机会 - 每个StockCard自带可展开K线图（懒加载） - 分页：每页20只，共5页

顶部统计摘要： - 总只数（100） - 买点信号数量（买入/卖出/持有分布） - 上涨/下跌数量 - 热点板块TOP3（按出现频率最高的板块）

5. 执行计划

详见：[热点个股追踪 — 执行计划](#)

6. 附录

6.1 替代方案

- **自建热点扫描：**需要全市场K线数据+成交量排名+涨幅排名，计算量大且与同花顺热度排名可能差异大。不如直接使用同花顺成熟的热度算法。
- **仅展示原始数据：**不做3L分析的话，就是同花顺的翻版，没有价值增量。

6.2 开放问题

1. 后端100只逐个调用 `get_stock_card()` 的性能问题 — 预估3-5秒，可以做批量并行优化
2. 同花顺API稳定性 — 如果API变或限制，考虑缓存上次结果做降级
3. 是否需要加入1h（小时级）热股数据

6.3 文件清单

新增：		
server/backend/api/hot_stocks.py	# 热股API路由+服务	
server/frontend/src/pages/HotStocks.tsx	# 热股追踪页面	
server/frontend/src/pages/HotStocks.css	# 热股页面样式	
修改：		
server/server.py	# 注册API路由+SPA路由	
server/frontend/src/App.tsx	# 注册页面路由	
server/frontend/src/components/NavBar.tsx	# 添加侧边栏导航项	

6.4 变更日志

版本	日期	变更内容
v1	2026-06-04	初稿 — 完成整体设计